

Qué es un diagrama de secuencia y para qué sirve

Un diagrama de secuencia es un tipo de diagrama de interacción del lenguaje UML que sirve para modelar cómo se comunican entre sí los objetos de un sistema a lo largo del tiempo. La idea central es mostrar en orden (por eso secuencia), qué mensajes se intercambian, en qué momento se envían y cómo se relacionan unos con otros. En inglés también se les conoce como sequence diagrams, event-trace diagrams o event scenarios.

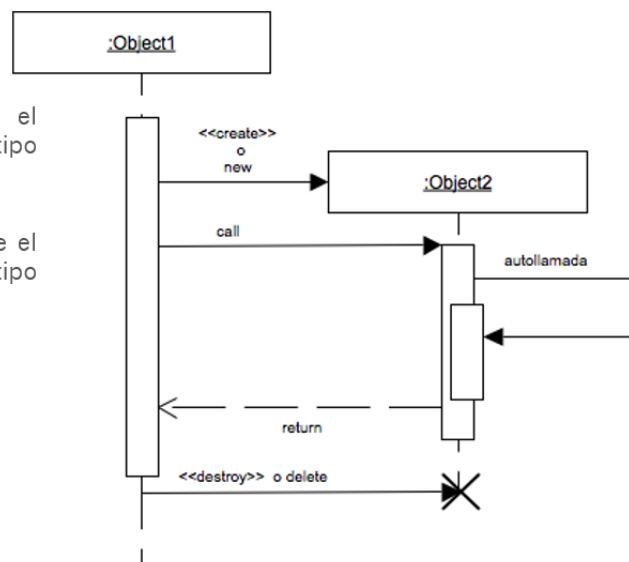
Tiene dos dimensiones la dimensión temporal, que se representa de manera vertical y va de arriba hacia abajo (lo que está más arriba ocurre antes) y la dimensión de instancias, que se representa de manera horizontal. El orden relativo en que se colocan los objetos en el eje horizontal no tiene importancia; lo único que importa de verdad es el flujo temporal de los mensajes. Su utilidad es doble. Por un lado, se usa para modelar cada caso de uso del sistema: muestra paso a paso cómo el conjunto de objetos colabora para que ese caso de uso ocurra. Por otro, sirve para describir métodos particulares de una clase, detallando qué llamadas hace y a quién. Mientras que un diagrama de casos de uso ofrece una vista de negocio del escenario, el diagrama de secuencia ya entra en los detalles de implementación: qué objetos y clases concretos participan, y qué mensajes intercambian.

Introducción a UML:

Diagramas de comportamiento

Diagramas de secuencia.

- Se puede representar:
 - La creación de objetos mediante el estereotipo `<<create>>` (estereotipo opcional)
 - La destrucción de objetos mediante el estereotipo `<<destroy>>` (estereotipo opcional)



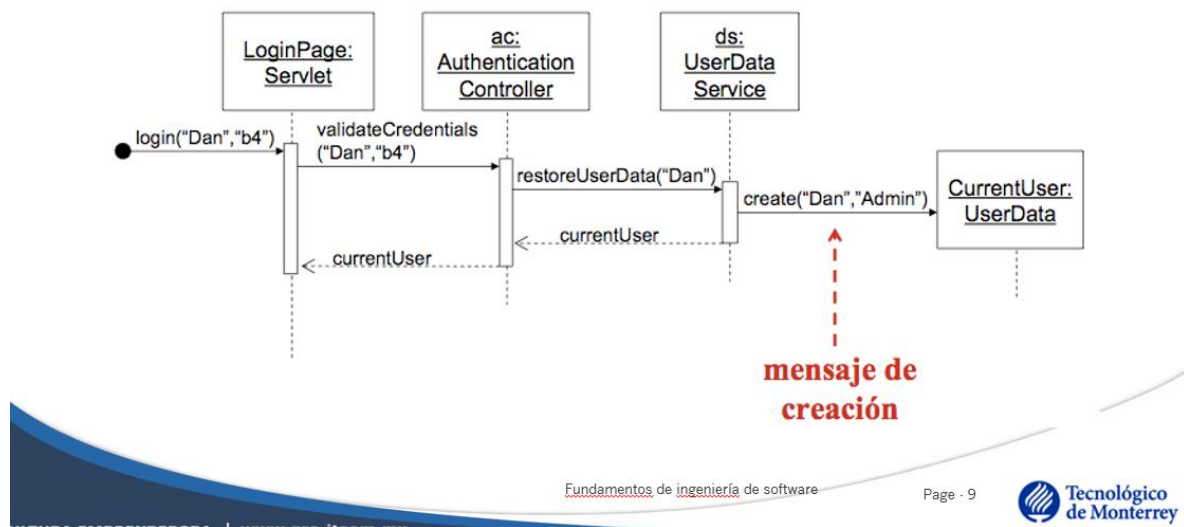
La línea de vida es una línea discontinua vertical que cae desde cada objeto y representa su existencia durante el tiempo del diagrama. Hay un puñado de errores que aparecen una y otra vez. Confundir línea de vida con foco de control es el más típico la línea es discontinua y representa existencia, el rectángulo representa actividad.

Introducción a UML:

Diagramas de comportamiento

Diagramas de secuencia.

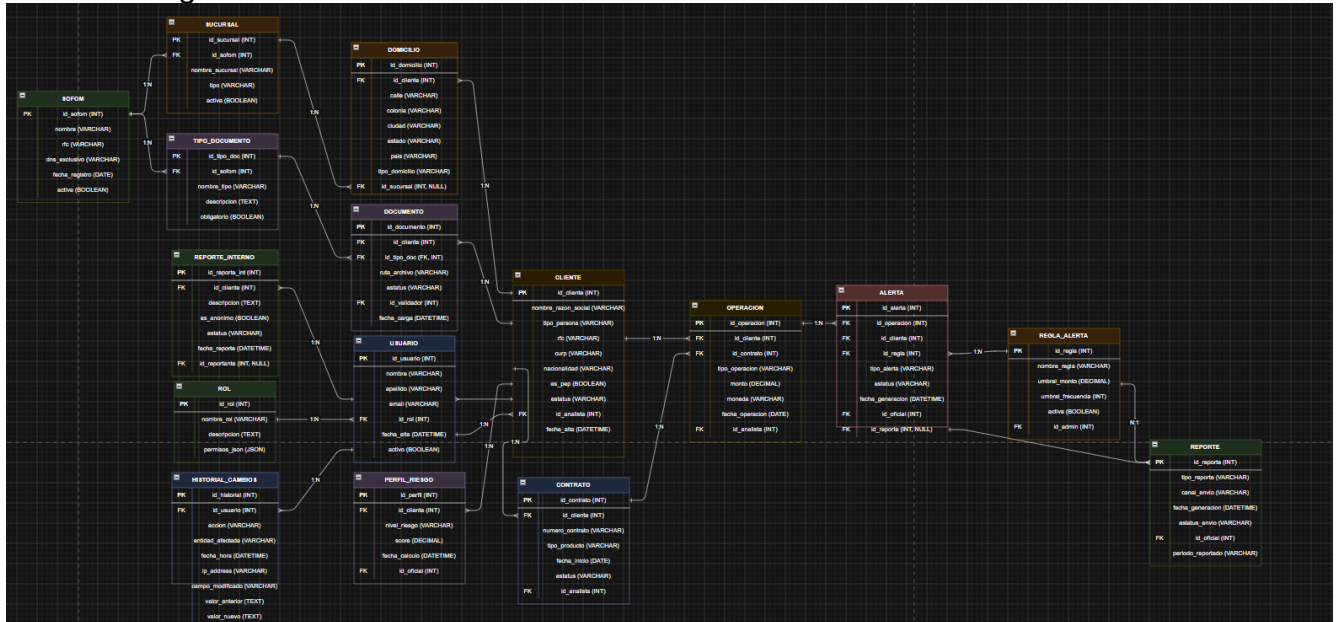
- Ejemplo:



Sus elementos básicos son la línea de vida (línea discontinua vertical que representa la existencia del objeto) y el foco de control (rectángulo delgado superpuesto que representa el periodo en que el objeto está ejecutando una acción). También se pueden marcar la creación de objetos con el estereotipo `<<create>>` y la destrucción con `<<destroy>>`, ambos opcionales. Hay tres tipos de mensajes: el síncrono (flecha de cabeza llena) es una llamada a método donde el emisor queda bloqueado hasta recibir respuesta; el asíncrono (flecha de cabeza abierta) termina inmediatamente y crea un nuevo hilo de ejecución; y el retorno (flecha discontinua) es la respuesta a una llamada y puede omitirse si queda claro por el fin de la activación.

Para nuestros avances de esta UF nos ayudaron los diagramas a cada paso, por ser una manera gráfica y visual de plasmar las ideas en algo gráfico y es como pudimos modelar el flujo que podría llevar la base de datos y el flujo que podría

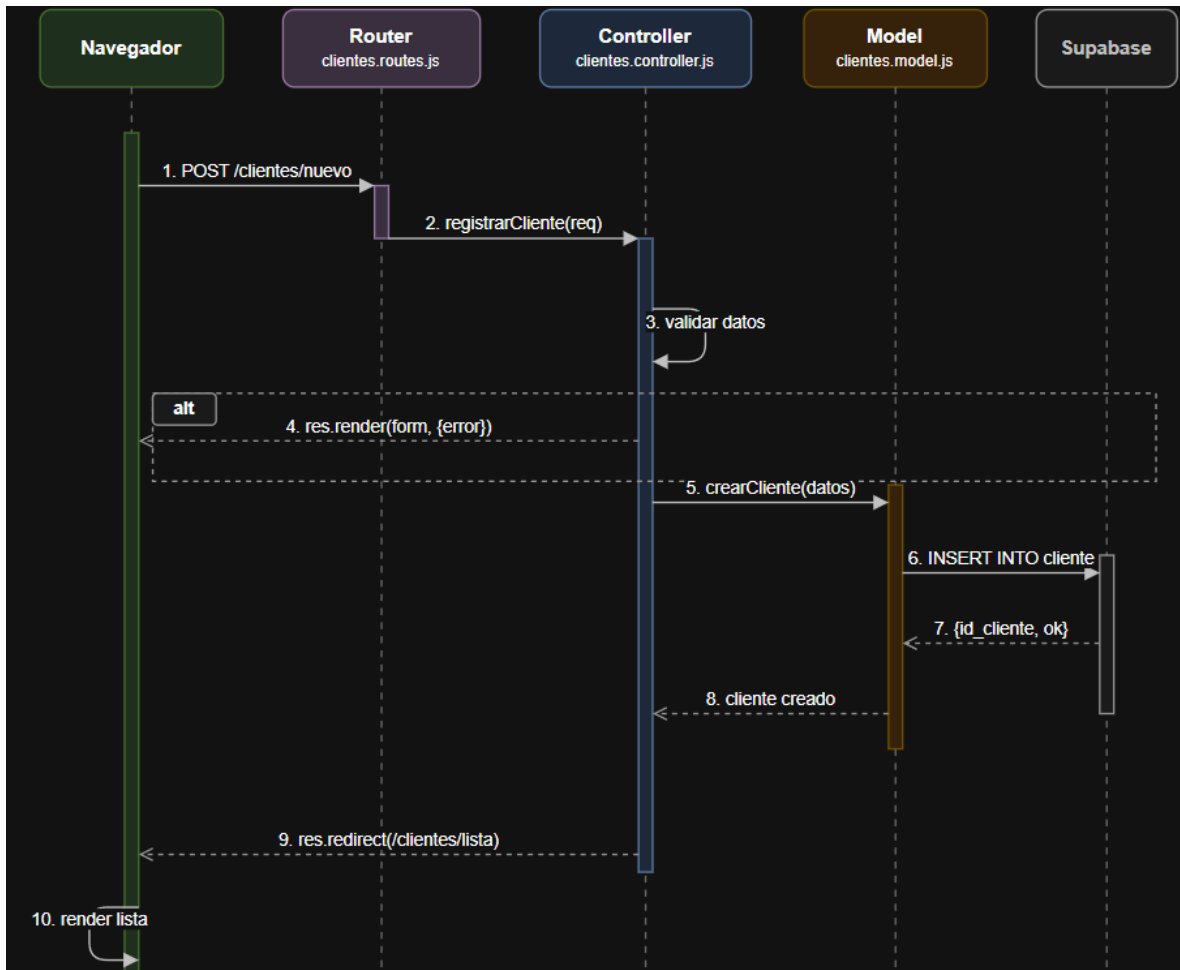
llevar el código. Es por ello por lo que dan una excelente planificación para lo que sería el código a futuro.



En nuestro primer modelo de Modelo Entidad Relación:

https://drive.google.com/file/d/1HLTLGcmOpfUMsNul7eoR6_utivvwzr2/view?usp=sharing

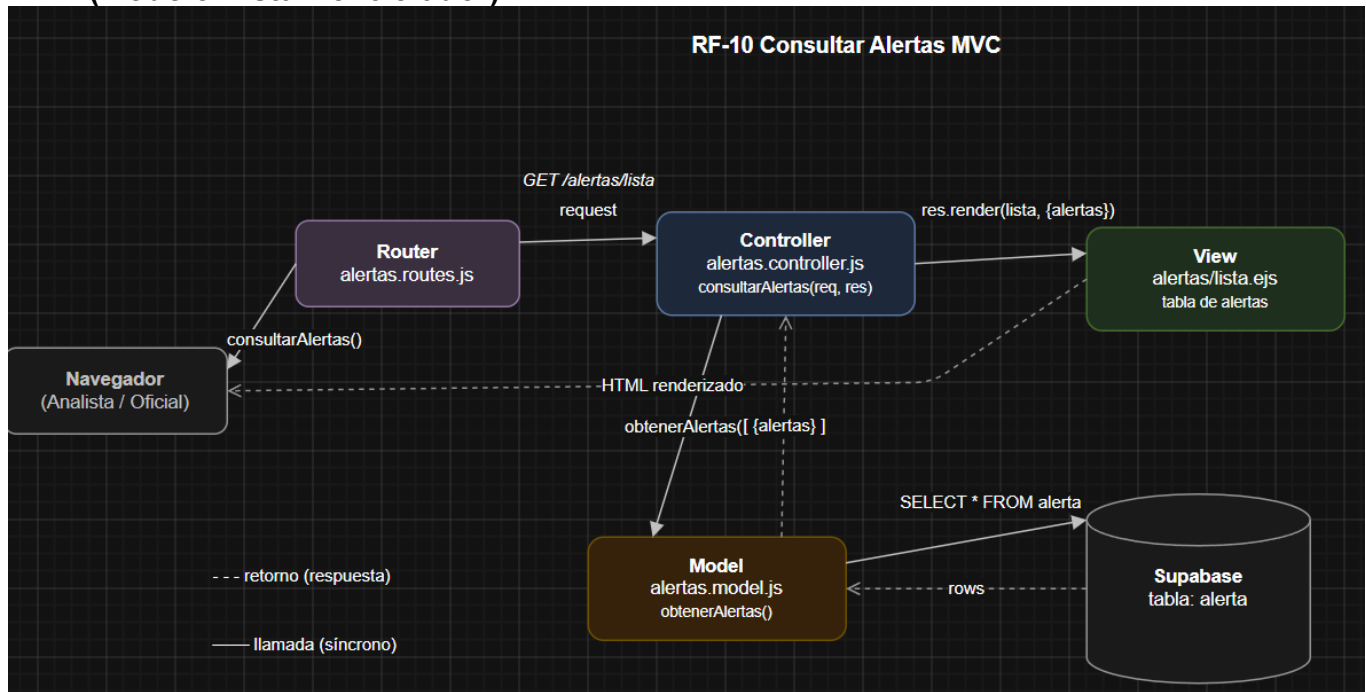
Que dejo adjuntado aquí y esta adjuntado en nuestro “Avance 2” de la materia. Este me ayudó en el proyecto porque funcionó como un elemento de diseño previo a la programación, ya que me permitió visualizar cómo se iba a organizar la base de datos y cómo se relacionan las partes principales del sistema, como SOFOM, cliente, operación, alerta, reporte, documento y perfil de riesgo. A partir de este MER pude entender mejor qué información debía guardar cada tabla, cuáles eran sus llaves primarias y foráneas, y cómo se conectaban los procesos del sistema de prevención de lavado de dinero. Además, me sirvió como guía para traducir el diseño a scripts de PostgreSQL y para evitar crear tablas aisladas sin una estructura clara. Esta planificación ayuda a posteriormente evitar menos errores y evitar hacer código ‘quirúrgico’, de intentar cambiar o añadir algo que no estaba planificado y aunque es posible, es bastante mas complicado que de haberlo hecho desde un principio al haber estado visualizado o planeado.



El diagrama de secuencia que hicimos como una especie de borrador lo que modela es el flujo del requisito funcional RF-01 de Registrar al Cliente, que es uno de los requisitos ya implementados en el sistema CoreData. Lo que se buscó al hacerlo fue representar el patrón de arquitectura MVC que usa el proyecto, donde cada participante del diagrama corresponde a un archivo real: clientes.routes.js como Router, clientes.controller.js como Controller, clientes.model.js como Model, y el navegador como la capa de Vista. De esta forma el diagrama no solo está mostrando la comunicación entre los objetos, sino que funciona como un plano de la implementación real, dejando claro qué responsabilidad tiene cada capa antes de ponerse a escribir código.

El diagrama fue útil para identificar que el Controller es el único que valida los datos antes de llamar al Model, lo que evitó duplicar lógica de validación y de código, algo en lo que se nos ha hecho hincapié con la suficiente insistencia de no cometer dicho error. También ayudó a definir con claridad qué retorna cada capa: el Model regresa el objeto creado, el Controller decide si redirige o renderiza un error y la Vista nunca accede directamente a los datos. Ese nivel de detalle es lo que después se tradujo en la estructura de archivos que se puede ver en el Avance 5, donde routes, controllers, models y views tienen responsabilidades separadas y bien definidas.

MVC (Modelo Vista Controlador)



Para el modelo de este modelo de vista controlador, el diagrama de componentes del RF-10 muestra cómo el sistema va a consultar las alertas generadas siguiendo la misma arquitectura que guió el desarrollo del proyecto. Cuando el Analista o el Oficial de Cumplimiento accede a la lista de alertas, el navegador hace un GET a `/alertas/lista` que el Router recibe y delega al Controller, este llama a `obtenerAlertas()` en el Model, el Model ejecuta el query contra Supabase y regresa los datos al Controller, que finalmente renderiza `alertas/lista.ejs` con la información y la devuelve al navegador. Este diagrama fue útil antes de programar porque dejó claro que el Controller nunca toca la base de datos directamente y no la debe de tocar y que la Vista nunca recibe datos sin pasar por él, y esto evitó tener que reescribir lógica cuando se conectó Supabase en avances posteriores.